

LAB PROGRAM TOPIC 3-1

SHAHANAJ C

192372292

**SIMATS Online Programming Lab**

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292A

Language: **Python** C C++ Java

QUESTIONS

Q1 Sequential Search 1 mark

Q2 Sequential Search 1 mark

Q3 Sequential Search 1 mark

Q4 Sequential Search 1 mark

Q5 Sequential Search 1 mark

Q6 Brute Force String Matching 1 mark

Q7 Brute Force String Matching 1 mark

Q8 Brute Force String Matching 1 mark

Q9 Brute Force String Matching 1 mark

Q10 Brute Force String Matching 1 mark

10/10 answered

Q1 Sequential Search 1 mark
Implement Sequential Search and analyze its performance.
Input:
Array:
45, 23, 67, 12, 89, 34, 56, 78, 90, 11,
29, 73, 18, 64, 37
Search Keys:
73
18
100
Tasks:
- Show each comparison.
- Count comparisons for every search.
- Identify best, average, and worst cases.
- Determine:
- Best-case complexity
- Average-case complexity
- Worst-case complexity
- Space complexity

Your Code

```
def sequential_search(arr, key):  
    comparisons = 0  
    found = False  
    index = -1  
    print(f"Searching for key: {key}")  
    for i in range(len(arr)):  
        comparisons += 1  
        print(f"Comparison {comparisons}: Is {arr[i]} == {key}? ", end="")  
        if arr[i] == key:  
            print("Yes!")  
            found = True  
            index = i  
            break  
    return found, index, comparisons
```


Input
stdin input
Output
--- Sequential Search Analysis ---
Searching for key: 73
Comparison 1: Is 45 == 73? No
Comparison 2: Is 23 == 73? No
Comparison 3: Is 67 == 73? No
Comparison 4: Is 12 == 73? No
Comparison 5: Is 89 == 73? No
Comparison 6: Is 34 == 73? No
Comparison 7: Is 56 == 73? No
Comparison 8: Is 78 == 73? No
Comparison 9: Is 90 == 73? No
Comparison 10: Is 11 == 73? No
Comparison 11: Is 29 == 73? No
Comparison 12: Is 73 == 73? Yes!
Found at index 12
Best-case complexity: O(1)
Average-case complexity: O(n/2)
Worst-case complexity: O(n)
Space complexity: O(1)

**SIMATS Online Programming Lab**

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292A

LAB PROGRAM -TOPIC 3-1

Language: **Python** C C++ Java

QUESTIONS

Q1 Sequential Search 1 mark

Q2 Sequential Search 1 mark

Q3 Sequential Search 1 mark

Q4 Sequential Search 1 mark

Q5 Sequential Search 1 mark

Q6 Brute Force String Matching 1 mark

Q7 Brute Force String Matching 1 mark

Q8 Brute Force String Matching 1 mark

Q9 Brute Force String Matching 1 mark

Q10 Brute Force String Matching 1 mark

10/10 answered

Q2 Sequential Search 1 mark
Write a program to perform Sequential Search on a 2-D matrix.
Input:
12 8 15
5 18 27
9 11 24
Key = 24
Output:
Element found at Row 3 Column 3

Your Code

```
def main():  
    matrix = []  
    for _ in range(3):  
        matrix.append(list(map(int, input().split())))  
    line = input().strip()  
    if "==" in line:  
        key = int(line.split("==")[1].strip())  
    else:  
        key = int(line)  
    for i in range(len(matrix)):  
        for j in range(len(matrix[i])):  
            if matrix[i][j] == key:  
                print(f"Element found at Row {i+1} Column {j+1}")  
                return  
    print("Element not found")  
if __name__ == "__main__":  
    main()
```


Input
12 8 15
5 18 27
9 11 24
KEY= 24
Output
Element found at Row 3 Column 3

Submitted

LAB PROGRAM -TOPIC 3-1

[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

- Q1
Sequential Search
1 mark
- Q2
Sequential Search
1 mark
- Q3
Sequential Search
1 mark
- Q4
Sequential Search
1 mark
- Q5
Sequential Search
1 mark
- Q6
Brute Force String Matching
1 mark
- Q7
Brute Force String Matching
1 mark
- Q8
Brute Force String Matching
1 mark
- Q9
Brute Force String Matching
1 mark
- Q10
Brute Force String Matching
1 mark

10/10 answered

Q3 Sequential Search

1 mark

Implement Sequential Search for string data.

Input:

Names:
Anu, Bala, Charan, Deepa, Esha, Farhan

Search Name:
Deepa

Output:
Name found at position 4

Your Code

```
1 names = ["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"]
2 search_name = "Deepa"
3
4 position = -1
5 for i in range(len(names)):
6     if names[i] == search_name:
7         position = i + 1
8         break
9
10 if position != -1:
11     print("Name found at position", position)
12 else:
13     print("Name not found")
```

[Submitted](#)

Input

Deepa

Output

Name found at position 4

LAB PROGRAM -TOPIC 3-1

[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

- Q1
Sequential Search
1 mark
- Q2
Sequential Search
1 mark
- Q3
Sequential Search
1 mark
- Q4
Sequential Search
1 mark
- Q5
Sequential Search
1 mark
- Q6
Brute Force String Matching
1 mark
- Q7
Brute Force String Matching
1 mark
- Q8
Brute Force String Matching
1 mark
- Q9
Brute Force String Matching
1 mark
- Q10
Brute Force String Matching
1 mark

10/10 answered

Q4 Sequential Search

1 mark

Write a program to perform Sequential Search on an array of student register numbers.

Input:

Register Numbers:
101, 102, 103, 104, 105, 106

Search Key: 104

Output:
Register Number found at position 4

Your Code

```
1 def sequential_search(arr, key):
2     for i in range(len(arr)):
3         if arr[i] == key:
4             return i + 1
5     return -1
6
7 registers = [101, 102, 103, 104, 105, 106]
8 search_key = 104
9
10 pos = sequential_search(registers, search_key)
11
12 if pos != -1:
13     print(f"Register Number found at position (pos)")
14 else:
15     print("Register Number not found")
```

[Submitted](#)

Input

stdin input

Output

Register Number found at position 4



SIMATS
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

 CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 3-1

Back

Language: Python C C++ Java

QUESTIONS

Q1
Sequential Search
1 mark

Q2
Sequential Search
1 mark

Q3
Sequential Search
1 mark

Q4
Sequential Search
1 mark

Q5
Sequential Search
1 mark

Q6
Brute Force String Matching
1 mark

Q7
Brute Force String Matching
1 mark

Q8
Brute Force String Matching
1 mark

Q9
Brute Force String Matching
1 mark

Q10
Brute Force String Matching
1 mark

10/10 answered

Q5 Sequential Search1 mark

Implement Sentinel Sequential Search and compare it with ordinary Sequential Search.

Input:
Array: 14, 9, 22, 35, 18, 41, 27
Key: 18

Output:
Position found
Comparison count

Your Code

```
1 def ordinary_sequential_search(arr, key):
2     comp = 0
3     for i in range(len(arr)):
4         comp += 1
5         if arr[i] == key:
6             return i, comp
7     return -1, comp
8
9 def sentinel_sequential_search(arr, key):
10    n = len(arr)
11    comp = 0
12
13    last = arr[-1]
14    arr[-1] = key
15
16    i = 0
17    while True:
18        comp += 1
19        if arr[i] == key:
20            break
21        i += 1
22
23    arr[-1] = last
24
25    if i < n - 1 or last == key:
26        return i, comp
27    else:
28        return -1, comp
```

Input

stdin input

Output

Ordinary Sequential Search:
Position = 4 Comparisons = 5
Sentinel Sequential Search:
Position = 4 Comparisons = 5



SIMATS
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

 CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 3-1

Back

Language: Python C C++ Java

QUESTIONS

Q1
Sequential Search
1 mark

Q2
Sequential Search
1 mark

Q3
Sequential Search
1 mark

Q4
Sequential Search
1 mark

Q5
Sequential Search
1 mark

Q6
Brute Force String Matching
1 mark

Q7
Brute Force String Matching
1 mark

Q8
Brute Force String Matching
1 mark

Q9
Brute Force String Matching
1 mark

Q10
Brute Force String Matching
1 mark

10/10 answered

Q6 Brute Force String Matching1 mark

Write a complete program for Brute Force String Matching and perform complexity analysis.

Input:
- Text = "TTATAGATCTGATTCTTTATAGATCTCCTATTCTT"
- Pattern = "TATCTT"

Tasks:
- Find all occurrences of the pattern.
- Count total comparisons.
- Display the shifting process.
- Analyze:
- Best-case complexity
- Worst-case complexity
- Space complexity

Your Code

```
1 def brute_force_string_matching(text, pattern):
2     n = len(text)
3     m = len(pattern)
4     total_comparisons = 0
5     occurrences = []
6
7     print(f"Text: {text}")
8     print(f"Pattern: {pattern}")
9     print("-" * 30)
10    for i in range(n - m + 1):
11        match = True
12        print(f"Shift {i}: Checking alignment at index {i}")
13
14        for j in range(m):
15            total_comparisons += 1
16            if text[i + j] != pattern[j]:
17                match = False
18                print(f" Mismatch at text[{i+j}] ('{text[i+j]}') and p
19                break
20            else:
21                print(f" Match at text[{i+j}] ('{text[i+j]}') and pat
22
```

Input

stdin input

Output

Text:
TTATAGATCTGATTCTTTATAGATCTCCT
ATTCTT
Pattern: TATCTT

Shift 0: Checking alignment at



SIMATS Online Programming Lab
DAA PROGRAMMING


CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM - TOPIC 3-1
Back

Language:
Python
C
C++
Java

QUESTIONS

Q1
Sequential Search
1 mark

Q2
Sequential Search
1 mark

Q3
Sequential Search
1 mark

Q4
Sequential Search
1 mark

Q5
Sequential Search
1 mark

Q6
Brute Force String Matching
1 mark

Q7
Brute Force String Matching
1 mark

Q8
Brute Force String Matching
1 mark

Q9
Brute Force String Matching
1 mark

Q10
Brute Force String Matching
1 mark

Q7 Brute Force String Matching
1 mark

Implement Brute Force String Matching and display every alignment of the pattern with the text.

Input:

- Text = "ABCDABCAABCA"
- Pattern = "ABCA"

Output:

- Alignment number
- Matching result
- Pattern occurrence positions

Your Code

```

1 def brute_force_string_matching(text, pattern):
2     n = len(text)
3     m = len(pattern)
4     occurrences = []
5
6     print(f"Text: {text}")
7     print(f"Pattern: {pattern}\n")
8     for i in range(n - m + 1):
9         alignment_num = i + 1
10        current_window = text[i : i + m]
11        if current_window == pattern:
12            print(f"Alignment {alignment_num}: Match found at position")
13            occurrences.append(i)
14        else:
15            print(f"Alignment {alignment_num}: No match")
16
17    print("\n--- Results ---")
18    print(f"Pattern occurrence positions: {occurrences}")
19    text_input = "ABCDABCAABCA"
20    pattern_input = "ABCA"
21    brute_force_string_matching(text_input, pattern_input)

```

Input

stdin input

Output

```

Text: ABCDABCAABCA
Pattern: ABCA
Alignment 1: Match found at position 0
Alignment 2: No match

```



SIMATS Online Programming Lab
DAA PROGRAMMING


CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM - TOPIC 3-1
Back

Language:
Python
C
C++
Java

QUESTIONS

Q1
Sequential Search
1 mark

Q2
Sequential Search
1 mark

Q3
Sequential Search
1 mark

Q4
Sequential Search
1 mark

Q5
Sequential Search
1 mark

Q6
Brute Force String Matching
1 mark

Q7
Brute Force String Matching
1 mark

Q8
Brute Force String Matching
1 mark

Q9
Brute Force String Matching
1 mark

Q10
Brute Force String Matching
1 mark

Q8 Brute Force String Matching
1 mark

Write a program to compare the execution of Brute Force String Matching for:

- Successful search
- Unsuccessful search

Input:

- Text = "PROGRAMMINGLAB"
- Pattern 1 = "LAB"
- Pattern 2 = "TEST"

Display the comparison count for both cases.

Your Code

```

1 def brute_force_count_comparisons(text, pattern):
2     n = len(text)
3     m = len(pattern)
4     comparisons = 0
5
6     for i in range(n - m + 1):
7         j = 0
8         while j < m:
9             comparisons += 1
10            if text[i + j] != pattern[j]:
11                break
12            j += 1
13        if j == m:
14            return True, comparisons
15
16    return False, comparisons
17
18 text = "PROGRAMMINGLAB"
19 pattern1 = "LAB"
20 pattern2 = "TEST"
21
22 found1, comp1 = brute_force_count_comparisons(text, pattern1)
23 found2, comp2 = brute_force_count_comparisons(text, pattern2)
24
25 print("Brute Force String Matching Comparison Count")
26 print(f"Text = {text}")
27 print(f"Pattern 1 = {pattern1} (Successful Search): {found1}, {comp1}")
28 print(f"Pattern 2 = {pattern2} (Unsuccessful Search): {found2}, {comp2}")

```

Input

stdin input

Output

```

Brute Force String Matching
Comparison Count
Text = PROGRAMMINGLAB
Pattern 1 = LAB (Successful Search)
Found: True

```

10/10 answered

LAB PROGRAM -TOPIC 3-1

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Sequential Search

1 mark

Q2

Sequential Search

1 mark

Q3

Sequential Search

1 mark

Q4

Sequential Search

1 mark

Q5

Sequential Search

1 mark

Q6

Brute Force String Matching

1 mark

Q7

Brute Force String Matching

1 mark

Q8

Brute Force String Matching

1 mark

Q9

Brute Force String Matching

1 mark

Q10

Brute Force String Matching

1 mark

10/10 answered

Q9 Brute Force String Matching

1 mark

Implement Brute Force String Matching for case-insensitive searching.

Input:

- Text = "DataStructuresAndAlgorithms"
- Pattern = "ALGORITHMS"

Output:

- Pattern found position

Your Code

```
1 def brute_force_case_insensitive(text, pattern):
2     text_lower = text.lower()
3     pattern_lower = pattern.lower()
4
5     n = len(text_lower)
6     m = len(pattern_lower)
7     for i in range(n - m + 1):
8         match = True
9         for j in range(m):
10             if text_lower[i + j] != pattern_lower[j]:
11                 match = False
12                 break
13
14         if match:
15             return i
16
17     return -1
18
19 text = "DataStructuresAndAlgorithms"
20 pattern = "ALGORITHMS"
21 result = brute_force_case_insensitive(text, pattern)
22
23 if result != -1:
24     print(result)
25 else:
26     print("Pattern not found")
```

Submitted

Input

stdin input

Output

17

LAB PROGRAM -TOPIC 3-1

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Sequential Search

1 mark

Q2

Sequential Search

1 mark

Q3

Sequential Search

1 mark

Q4

Sequential Search

1 mark

Q5

Sequential Search

1 mark

Q6

Brute Force String Matching

1 mark

Q7

Brute Force String Matching

1 mark

Q8

Brute Force String Matching

1 mark

Q9

Brute Force String Matching

1 mark

Q10

Brute Force String Matching

1 mark

10/10 answered

Q10 Brute Force String Matching

1 mark

Write a program that searches for a pattern in a text and reports the first occurrence only using Brute Force String Matching.

Input:

- Text = "COMPUTERSCIENCE"
- Pattern = "SCI"

Output:

- First occurrence position
- Number of comparisons

Your Code

```
1 def brute_force_string_matching(text, pattern):
2     n = len(text)
3     m = len(pattern)
4     comparisons = 0
5     first_occurrence = -1
6
7     for i in range(n - m + 1):
8         match = True
9         for j in range(m):
10             comparisons += 1
11             if text[i + j] != pattern[j]:
12                 match = False
13                 break
14
15         if match:
16             first_occurrence = i
17             break
18
19     return first_occurrence, comparisons
20
21 text = "COMPUTERSCIENCE"
22 pattern = "SCI"
23 position, total_comparisons = brute_force_string_matching(text, pattern)
24
25 print(f"First occurrence position: {position}")
26 print(f"Number of comparisons: {total_comparisons}")
```

Input

stdin input

Output

```
First occurrence position: 8
Number of comparisons: 11
```